

LAPORAN PRAKTIKUM

PEMROGRAMAN WEB – GIK1JAB3

Modul 12 - Membuat Aplikasi React Blog Post Menggunakan API

CHALISA GAVRILLA AZHARI

707012500127

D4 SIKC 49-04

PROGRAM STUDI D4 SISTEM INFORMASI KOTA CERDAS

FAKULTAS ILMU TERAPAN

UNIVERSITAS TELKOM

BANDUNG

2026

A. Tujuan Praktikum

- a. Mahasiswa mampu menginisialisasi proyek *frontend* baru menggunakan Vite dan React JS.
- b. Mahasiswa mampu mengintegrasikan Tailwind CSS 3 ke dalam ekosistem React-Vite untuk membangun antarmuka yang responsif.
- c. Mahasiswa memahami dan mampu mengimplementasikan *Global State Management* menggunakan React Context API untuk proses autentikasi (Login/Logout).
- d. Mahasiswa mampu menerapkan *Client-Side Routing* dan proteksi halaman (*Protected Route*) menggunakan react-router-dom.
- e. Mahasiswa mampu mengintegrasikan aplikasi *frontend* React dengan *backend* REST API pihak ketiga menggunakan library Axios dan memanfaatkan fitur Vite Proxy.

B. Alat dan Bahan

- a. **Perangkat Keras:** Laptop / Komputer PC.
- b. **Sistem Operasi:** Windows / macOS / Linux.
- c. **Perangkat Lunak:**
- d. Node.js & npm (Node Package Manager).
- e. Git Bash / Terminal bawaan.
- f. Visual Studio Code (VS Code) sebagai *Code Editor*.
- g. Web Browser (Google Chrome / Mozilla Firefox).
- h. Postman (untuk pengujian API).
- i. XAMPP (Apache) untuk menjalankan REST API CodeIgniter 3.

C. Langkah-langkah praktikum

1. Pastikan REST API bisa diakses
2. Buat folder Pekan-12 dalam folder WebPro4904 untuk memudahkan pengumpulan pada github.
3. Buka gitbash lagi pada folder, Install Library Tambahan,
4. Lalu ikuti model hingga selesai

D. Hasil dan Dokumentasi Program

1. src/components/DefaultAvatar.jsx

```
1  const DefaultAvatar = ({ className = "w-7 h-7" }) => ( // Default avatar component with customizable size
2    <div className={` ${className} rounded-full bg-gray-300 flex items-center justify-center flex-shrink-0`} >
3      <svg className="w-4 h-4 text-gray-500" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
4        <path fillRule="evenodd" d="M10 9a3 3 0 10-6 3 3 0 00 6 3zm-7 9a7 7 0 114 0H3z" clipRule="evenodd"></path>
5      </svg>
6    </div>
7  );
8
9  export default DefaultAvatar; // Export the DefaultAvatar component for use in other parts of the application
10
```

2. src/components/Footer.jsx

[illegible]

3. src/components/Navbar.jsx

4. src/components/PostCard.jsx

```

1 import { Link } from 'react-router-dom'; // Link component for navigation
2 import DefaultAvatar from './DefaultAvatar'; // Default avatar component for user profile display
3
4 const PostCard = ({ post }) => {
5   const truncateText = (text, maxLength = 150) => {
6     if (!text) return '';
7     if (text.length <= maxLength) return text;
8     return text.substring(0, maxLength) + '...';
9   };
10
11   const formatDate = (dateString) => {
12     if (!dateString) return '14 days ago';
13     const date = new Date(dateString);
14     const now = new Date();
15     const diffDays = Math.floor((now - date) / (1000 * 60 * 60 * 24));
16     if (diffDays === 0) return 'Today';
17     if (diffDays === 1) return 'Yesterday';
18     return `${diffDays} days ago`;
19   };
20
21   return (
22     <article className="p-6 bg-white rounded-lg border border-gray-200 shadow-md hover:shadow-lg transition-shadow">
23       <div className="flex justify-between items-center mb-5 text-gray-500">
24         <span className="bg-blue-100 text-blue-800 text-xs font-medium inline-flex items-center px-2.5 py-0.5 rounded">
25           <svg className="mr-1 w-3 h-3" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
26             <path fillRule="evenodd" d="M2 5a2 2 0 012-2h8a2 2 0 012 2v10a2 2 0 002 2h2a2 2 0 012-2V5zm3
27             1h6v4H5V6zm6 6H5v2h6v-2z" clipRule="evenodd"></path>
28             <path d="M15 7h1a2 2 0 012 2v5.5a1.5 1.5 0 01-3 0V7z"></path>
29           </svg>
30           Article
31         </span>
32         <span className="text-sm flex items-center">
33           <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
34             <path fillRule="evenodd" d="M6 2a1 1 0 00-1 1v1H4a2 2 0 00-2 2v10a2 2 0 002 2h12a2 2 0 002-2V6a2 2
35             0 00-2-2h-1V3a1 1 0 00-2 0v1H7V3a1 1 0 00-1-1zm0 5a1 1 0 000 2h8a1 1 0 100-2h6z" clipRule="evenodd"></path>
36           </svg>
37           {formatDate(post.created_at)}
38         </span>
39       </div>
40       <h2 className="mb-2 text-2xl font-bold tracking-tight text-gray-900">
41         <Link to={`/${post.id}`} className="hover:text-blue-600 transition-colors">
42           {post.title}
43         </Link>
44       </h2>
45       <p className="mb-5 font-light text-gray-500">
46         {truncateText(post.article)}
47       </p>
48       <div className="flex justify-between items-center">
49         <div className="flex items-center space-x-3">
50           <DefaultAvatar />
51           <span className="font-medium text-gray-900">{post.author}</span>
52         </div>
53         <Link
54           to={`/${post.id}`}
55           className="inline-flex items-center font-medium text-blue-600 hover:text-blue-500 transition-colors"
56         >
57           Read more
58           <svg className="ml-2 w-4 h-4" fill="currentColor" viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
59             <path fillRule="evenodd" d="M10.293 3.293a1 1 0 011.414 0l6 6a1 1 0 001.414-6 6a1 1 0 01-1.414-1.
60             414l14.586 11H3a1 1 0 10-2 0h11.586l-4.293-4.293a1 1 0 00-1.414z" clipRule="evenodd"></path>
61           </svg>
62         </Link>
63       </div>
64     </article>
65   );
66 };
67
68 export default PostCard; // untuk mengeksport komponen PostCard agar dapat digunakan di bagian lain dari aplikasi
69

```

5. src/components/Protected.jsx



```
1 import { Navigate } from 'react-router-dom'; //untuk mengarahkan pengguna ke halaman login
2 // jika mereka tidak terautentikasi
3 import { useAuth } from '../context/AuthContext';
4 //untuk mengakses status autentikasi pengguna dari konteks AuthContext
5
6 const ProtectedRoute = ({ children }) => {
7   const { isAuthenticated, loading } = useAuth();
8
9   if (loading) {
10     return (
11       <div className="loading-container">
12         <div className="loading-spinner"></div>
13         <p>Loading...</p>
14       </div>
15     );
16   }
17
18   if (!isAuthenticated) {
19     return <Navigate to="/login" replace />;
20   }
21
22   return children;
23 };
24
25 export default ProtectedRoute;
26 //untuk mengekspor komponen ProtectedRoute agar dapat digunakan di bagian lain
27 // dari aplikasi, terutama untuk membungkus rute yang memerlukan autentikasi pengguna.
28
```

6. src/context/AuthContext.jsx

```

1 import { createContext, useContext, useState, useEffect } from 'react'; //untuk membuat konteks, mengakses konteks, dan mengelola state
2 // serta efek samping dalam komponen React
3 import { authAPI } from '../services/api'; //untuk mengimpor fungsi-fungsi yang berhubungan dengan autentikasi dari file api.js,
4 // seperti login dan logout, yang akan digunakan dalam konteks ini untuk mengelola status autentikasi pengguna.
5
6 const AuthContext = createContext(null); //untuk membuat konteks autentikasi yang akan digunakan untuk menyediakan informasi tentang status
7 // autentikasi pengguna dan fungsi-fungsi terkait ke seluruh aplikasi React.
8
9 export const AuthProvider = ({ children }) => {
10   const [user, setUser] = useState(null);
11   const [token, setToken] = useState(localStorage.getItem('token'));
12   const [loading, setLoading] = useState(true);
13
14   useEffect(() => { //untuk memeriksa status autentikasi pengguna saat komponen AuthProvider pertama kali dimuat. Fungsi checkAuth
15     // akan mencoba mengambil token dan data pengguna yang disimpan di localStorage. Jika ditemukan, token dan data pengguna
16     // akan disimpan dalam state. Setelah pemeriksaan selesai, state loading akan diatur ke false untuk menunjukkan bahwa proses pemeriksaan telah selesai.
17     const checkAuth = async () => {
18       const savedToken = localStorage.getItem('token');
19       const savedUser = localStorage.getItem('user');
20
21       if (savedToken && savedUser) {
22         setToken(savedToken);
23         setUser(JSON.parse(savedUser));
24       }
25       setLoading(false);
26     };
27
28     checkAuth(); //untuk memeriksa status autentikasi pengguna saat komponen AuthProvider pertama kali dimuat.
29     // Fungsi checkAuth akan mencoba mengambil token dan data pengguna yang disimpan di localStorage. Jika ditemukan,
30     // token dan data pengguna akan disimpan dalam state. Setelah pemeriksaan selesai, state loading akan diatur
31     // ke false untuk menunjukkan bahwa proses pemeriksaan telah selesai.
32   }, []);
33
34   const login = async (email, password) => {
35     try {
36       const response = await authAPI.login(email, password);
37       const { access_token, data: userData } = response.data;
38
39       localStorage.setItem('token', access_token);
40       localStorage.setItem('user', JSON.stringify(userData));
41
42       setToken(access_token);
43       setUser(userData);
44
45       return { success: true };
46     } catch (error) {
47       const message = error.response?.data?.message || 'Login gagal';
48       return { success: false, message };
49     }
50   };
51
52   const logout = async () => { //untuk mengelola proses logout pengguna.
53     // Fungsi logout akan mencoba memanggil API logout untuk memberitahu server
54     // bahwa pengguna ingin keluar. Setelah itu, token dan data pengguna akan dihapus
55     // dari localStorage, dan state token serta user akan diatur ke null untuk mencerminkan
56     // bahwa pengguna telah keluar dari aplikasi.
57     try {
58       await authAPI.logout();
59     } catch (error) {
60       console.error('Logout error:', error);
61     } finally {
62       localStorage.removeItem('token');
63       localStorage.removeItem('user');
64       setToken(null);
65       setUser(null);
66     }
67   };
68
69   const value = {
70     user,
71     token,
72     loading,
73     isAuthenticated: !!token,
74     login,
75     logout,
76   };
77
78   return (
79     <AuthContext.Provider value={value}>
80       {children}
81     </AuthContext.Provider>
82   );
83 };
84
85 export const useAuth = () => { //untuk membuat custom hook useAuth yang memungkinkan
86   // komponen lain untuk dengan mudah mengakses informasi autentikasi dan fungsi-fungsi terkait
87   // dari konteks AuthContext. Hook ini akan memeriksa apakah konteks tersedia dan melemparkan
88   // error jika digunakan di luar AuthProvider, memastikan bahwa hanya komponen yang berada dalam AuthProvider yang dapat mengakses informasi autentikasi.
89   const context = useContext(AuthContext);
90   if (!context) {
91     throw new Error('useAuth harus digunakan di dalam AuthProvider');
92   }
93   return context; //untuk membuat custom hook useAuth yang memungkinkan komponen lain
94   // untuk dengan mudah mengakses informasi autentikasi dan fungsi-fungsi terkait dari
95   // konteks AuthContext. Hook ini akan memeriksa apakah konteks tersedia dan melemparkan
96   // error jika digunakan di luar AuthProvider, memastikan bahwa hanya komponen yang berada
97   // dalam AuthProvider yang dapat mengakses informasi autentikasi.
98 };
99

```

7. src/pages/Login.jsx

```
1 import { useState } from 'react'; //untuk mengelola state lokal dalam komponen login, seperti email, password, error, dan loading.
2 import { useNavigate } from 'react-router-dom'; //untuk mengarahkan pengguna ke halaman lain setelah berhasil login, dalam hal
3 // ini ke halaman daftar postingan ("/posts").
4 import { useAuth } from '../context/AuthContext'; //untuk mengakses fungsi login dari konteks autentikasi yang telah dibuat
5 // sebelumnya, sehingga komponen Login dapat memanggil fungsi login untuk melakukan proses autentikasi pengguna saat form login disubmit.
6
7 const Login = () => { //Komponen Login yang menyediakan form untuk pengguna memasukkan email dan password mereka, serta menangani
8 // proses login dengan memanggil fungsi login dari konteks autentikasi. Komponen ini juga menampilkan pesan error jika login gagal
9 // dan mengarahkan pengguna ke halaman daftar postingan setelah berhasil login.
10   const [email, setEmail] = useState(''); //untuk menyimpan nilai email yang dimasukkan oleh pengguna dalam form login. State
11   // ini akan diperbarui setiap kali pengguna mengetik di input email, dan nilainya akan digunakan saat proses login untuk
12   // mengirimkan data autentikasi ke server.
13   const [password, setPassword] = useState(''); //untuk menyimpan nilai password yang dimasukkan oleh pengguna dalam
14   // form login. State ini akan diperbarui setiap kali pengguna mengetik di input password, dan nilainya akan digunakan
15   // saat proses login untuk mengirimkan data autentikasi ke server.
16   const [error, setError] = useState(''); //untuk menyimpan pesan error yang mungkin terjadi selama proses login.
17   // Jika login gagal, pesan error akan diperbarui dengan informasi yang relevan, dan pesan ini akan ditampilkan
18   // kepada pengguna untuk memberi tahu mereka tentang masalah yang terjadi saat mencoba login.
19   const [loading, setLoading] = useState(false); //untuk menyimpan status loading selama proses login berlangsung.
20   // State ini akan diatur ke true saat proses login dimulai dan kembali ke false setelah proses selesai, baik berhasil
21   // maupun gagal. Status loading ini dapat digunakan untuk menampilkan indikator loading kepada pengguna, seperti mengganti
22   // teks tombol login menjadi "Signing in..." atau menonaktifkan tombol login selama proses berlangsung untuk mencegah multiple submissions.
23
24   const { login } = useAuth(); //untuk mengakses fungsi login dari konteks autentikasi yang telah dibuat sebelumnya.
25   // Fungsi login ini akan dipanggil saat pengguna submit form login, dan akan menerima email dan password sebagai
26   // argumen untuk melakukan proses autentikasi dengan server. Hasil dari fungsi login akan digunakan untuk menentukan
27   // apakah login berhasil atau
28   const navigate = useNavigate(); //untuk mengarahkan pengguna ke halaman lain setelah berhasil login, dalam hal ini
29   // ke halaman daftar postingan ("/posts"). Fungsi navigate akan dipanggil dengan path tujuan setelah proses login berhasil,
30   // sehingga pengguna akan diarahkan ke halaman yang sesuai.
31
32   const handleSubmit = async (e) => { //untuk menangani event submit pada form login. Fungsi ini akan dipanggil ketika
33   // pengguna mengklik tombol login atau menekan Enter saat berada di dalam form. Fungsi ini akan mencegah perilaku
34   // default form, mengatur status loading, memanggil fungsi login dengan email dan password yang dimasukkan, dan
35   // menangani hasil dari proses login untuk menentukan apakah pengguna berhasil login
36     e.preventDefault();
37     setError('');
38     setLoading(true);
39
40     try {
41       const result = await login(email, password);
42
43       if (result.success) {
44         navigate('/posts');
45       } else {
46         setError(result.message);
47       }
48     } catch (err) {
49       console.error('Login error:', err);
50       setError(err.message || 'Terjadi kesalahan saat login. Pastikan server Laravel sudah berjalan.');
```

```

1      <form className="space-y-6" onSubmit={handleSubmit}>
2        <div>
3          <label htmlFor="email" className="block mb-2 text-sm font-medium text-gray-900">
4            Your email
5          </label>
6          <input
7            type="email"
8            name="email"
9            id="email"
10           value={email}
11           onChange={(e) => setEmail(e.target.value)}
12           className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-gray-900
13             placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-none transition-all"
14           placeholder="name@company.com"
15           required
16         />
17       </div>
18
19       <div>
20         <label htmlFor="password" className="block mb-2 text-sm font-medium text-gray-900">
21           Password
22         </label>
23         <input
24           type="password"
25           name="password"
26           id="password"
27           value={password}
28           onChange={(e) => setPassword(e.target.value)}
29           placeholder="*****"
30           className="w-full px-4 py-3 bg-gray-50 border border-gray-300 rounded-lg text-gray-900
31             placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-blue-500 outline-none transition-all"
32           required
33         />
34       </div>
35
36       <div className="flex items-center justify-between">
37         <div className="flex items-center">
38           <input
39             id="remember"
40             type="checkbox"
41             className="w-4 h-4 border border-gray-300 rounded bg-gray-50 focus:ring-2 focus:ring-blue-500"
42           />
43           <label htmlFor="remember" className="ml-2 text-sm text-gray-600">
44             Remember me
45           </label>
46         </div>
47         <a href="#" className="text-sm font-medium text-blue-600 hover:text-blue-500">
48           Forgot password?
49         </a>
50       </div>
51
52       <button
53         type="submit"
54         disabled={loading}
55         className="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 text-white
56           font-medium rounded-lg transition-colors focus:ring-4 focus:ring-blue-300
57           disabled:opacity-50 disabled:cursor-not-allowed"
58       >
59         {loading ? 'Signing in...' : 'Sign in'}
60       </button>
61
62       <p className="text-sm text-gray-600 text-center">
63         Don't have an account yet?{' '}
64         <a href="#" className="font-medium text-blue-600 hover:text-blue-500">
65           Sign up
66         </a>
67       </p>
68     </form>
69   </div>
70
71   <footer className="bg-white border-t border-gray-200 py-4">
72     <p className="text-center text-sm text-gray-500">
73       © 2024 Blog Posts. All rights reserved.
74     </p>
75   </footer>
76 </div>
77
78   );
79 };
80
81 export default Login; //untuk mengeksport komponen Login agar dapat digunakan di bagian
82 // lain dari aplikasi, terutama untuk digunakan sebagai rute login dalam aplikasi React.
83 // Komponen ini menyediakan form login dan menangani proses autentikasi pengguna.
84

```

8. `src/pages/PostDetail.jsx`

```

1 import { useState, useEffect } from 'react'; //untuk mengelola state lokal dalam komponen PostDetail,
2 // seperti menyimpan data post yang diambil dari API, status loading, dan pesan error. State ini akan diperbarui
3 // sesuai dengan hasil dari proses pengambilan data dan digunakan untuk menentukan apa yang ditampilkan kepada pengguna.
4 import { useParams, Link } from 'react-router-dom'; //untuk mengakses parameter id dari URL, yang digunakan
5 // untuk mengambil data post yang sesuai dari API. Selain itu, Link digunakan untuk membuat tautan yang memungkinkan
6 // pengguna kembali ke halaman daftar postingan setelah melihat detail sebuah post.
7 import { postsAPI } from '../services/api'; //untuk mengimpor fungsi-fungsi yang berhubungan dengan pengambilan data
8 // post dari API, seperti getById, yang akan digunakan untuk mengambil data post berdasarkan id yang diperoleh dari
9 // parameter URL.
10 import Navbar from '../components/Navbar'; //untuk mengimpor komponen Navbar yang akan digunakan untuk menampilkan
11 // navigasi di bagian atas halaman detail post. Navbar ini biasanya berisi tautan ke halaman lain dalam aplikasi,
12 // seperti daftar postingan, halaman login, atau halaman profil pengguna.
13 import Footer from '../components/Footer'; //untuk mengimpor komponen Footer yang akan digunakan untuk menampilkan
14 // bagian bawah halaman detail post
15 import DefaultAvatar from '../components/DefaultAvatar'; //untuk mengimpor komponen DefaultAvatar yang akan digunakan
16 // untuk menampilkan avatar default di halaman detail post, terutama jika data post tidak menyertakan informasi tentang
17 // avatar penulis. Kom
18
19 const PostDetail = () => { //Komponen PostDetail yang bertanggung jawab untuk menampilkan detail sebuah post berdasarkan
20 // id yang diperoleh dari parameter URL. Komponen ini akan mengambil data post dari API, mengelola status loading
21   const { id } = useParams(); //untuk mengakses parameter id dari URL, yang digunakan untuk mengambil data post
22   // yang sesuai dari API. Nilai id ini akan digunakan dalam fungsi fetchPost untuk memanggil API dan mendapatkan
23   // data post yang akan ditampilkan di halaman detail.
24   const [post, setPost] = useState(null); //untuk menyimpan data post yang diambil dari API. State ini akan
25   // diperbarui dengan data post yang berhasil diambil, dan nilai post akan digunakan untuk menampilkan informasi detail
26   // tentang post tersebut di halaman.
27   const [loading, setLoading] = useState(true); //untuk menyimpan status loading selama proses pengambilan data post
28   // berlangsung. State ini akan diatur ke true saat proses pengambilan data dimulai dan kembali ke false setelah
29   // proses selesai, baik berhasil maupun gagal. Status loading ini dapat digunakan untuk menampilkan indikator loading
30   // kepada pengguna, seperti spinner atau pesan "Loading...", selama data post sedang diambil dari API.
31   const [error, setError] = useState(''); //untuk menyimpan pesan error yang mungkin terjadi selama proses
32   // pengambilan data post. Jika terjadi kesalahan saat mencoba mengambil data dari API, pesan error akan diperbarui
33   // dengan informasi yang relevan, dan pesan ini akan ditampilkan kepada pengguna untuk memberi tahu mereka tentang
34   // masalah yang terjadi saat mencoba memuat detail post.
35
36   useEffect(() => {
37     fetchPost();
38   }, [id]);
39
40   const fetchPost = async () => {
41     try {
42       setLoading(true);
43       const response = await postsAPI.getById(id);
44       setPost(response.data.data);
45       setError('');
46     } catch (err) {
47       if (err.response?.status === 404) {
48         setError('Post tidak ditemukan.');

```

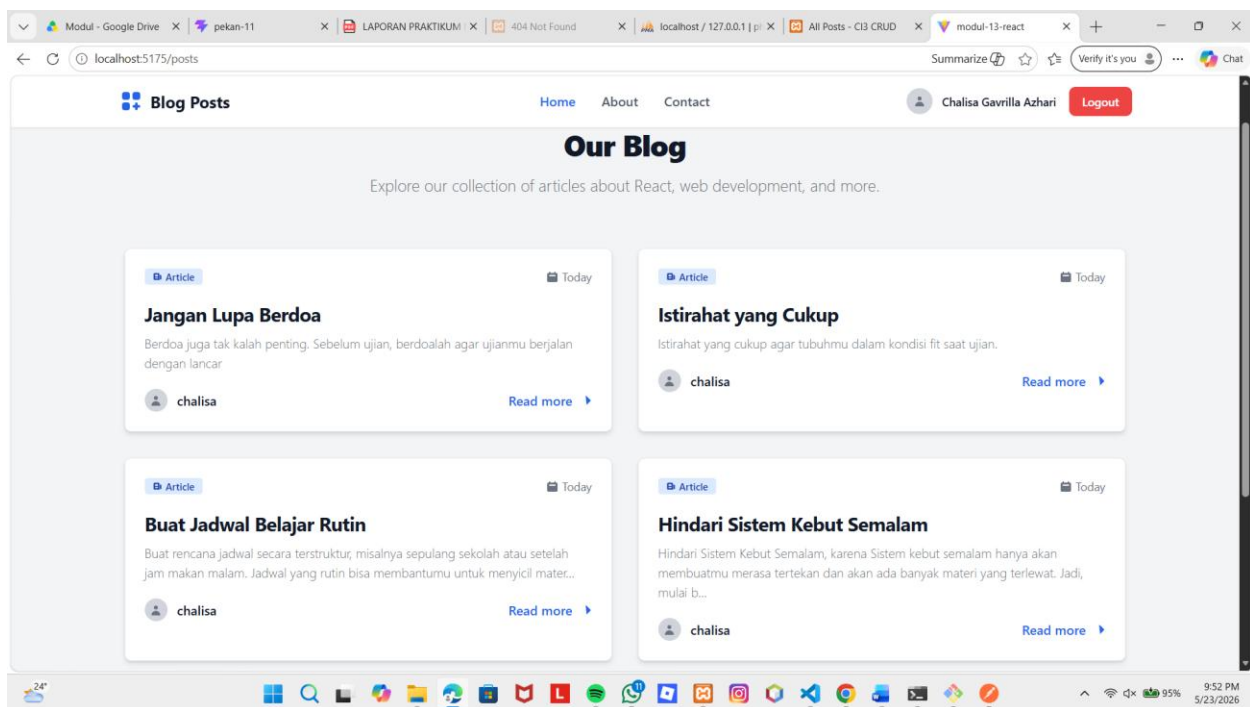
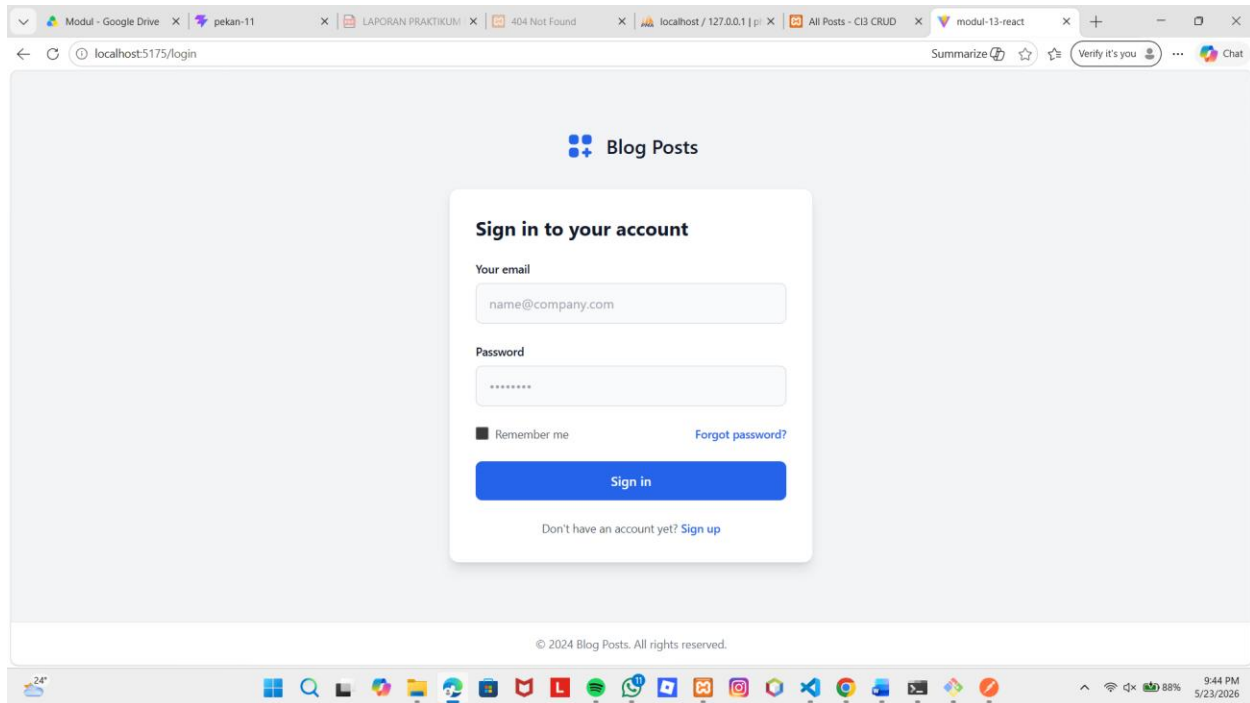

9. src/pages/PostList.jsx

```
1 import { useState, useEffect } from 'react'; //untuk mengelola state dan efek samping dalam komponen PostList,
2 // seperti menyimpan daftar posts, status loading, dan pesan error, serta untuk melakukan fetch data posts saat
3 // komponen pertama kali dimuat.
4 import { postsAPI } from '../services/api'; //untuk mengimpor fungsi-fungsi yang berhubungan dengan
5 // pengambilan data post dari API, seperti getAll, yang akan digunakan untuk mengambil daftar semua posts dari
6 // server Laravel.
7 import Navbar from '../components/Navbar'; //untuk mengimpor komponen Navbar yang akan digunakan untuk
8 // menampilkan navigasi di bagian atas halaman daftar postingan. Navbar ini biasanya berisi tautan ke
9 // halaman lain dalam aplikasi, seperti halaman login, halaman profil pengguna, atau halaman detail post
10 import Footer from '../components/Footer'; //untuk mengimpor komponen Footer yang akan digunakan untuk
11 // menampilkan bagian bawah halaman daftar postingan, biasanya berisi informasi hak cipta, tautan ke
12 // kebijakan privasi, atau informasi kontak.
13 import PostCard from '../components/PostCard'; //untuk mengimpor komponen PostCard yang akan digunakan
14 // untuk menampilkan setiap post dalam daftar postingan. Komponen PostCard biasanya berisi informasi singkat
15 // tentang post, seperti judul, cuplikan artikel, nama penulis, dan tanggal pembuatan, serta tautan ke halaman
16 // detail post
17
18 const PostList = () => { //Komponen Postlist yang bertanggung jawab untuk menampilkan daftar semua posts
19 // yang diambil dari API. Komponen ini akan mengelola status loading dan error selama proses pengambilan
20 // data, serta menampilkan daftar postingan menggunakan komponen PostCard untuk setiap post yang berhasil diambil.
21 const [posts, setPosts] = useState([]); //untuk menyimpan daftar posts yang diambil dari API. State ini
22 // akan diperbarui dengan data posts yang berhasil diambil, dan nilai posts akan digunakan untuk
23 // menampilkan daftar postingan di halaman menggunakan komponen PostCard untuk setiap post.
24 const [loading, setLoading] = useState(true); //untuk menyimpan status loading selama proses
25 // pengambilan data posts berlangsung. State ini akan diatur ke true saat proses pengambilan
26 // data dimulai dan kembali ke false setelah proses selesai, baik berhasil maupun gagal.
27 // Status loading ini dapat digunakan untuk menampilkan indikator loading kepada pengguna,
28 // seperti spinner atau pesan "Loading...", selama data posts sedang diambil dari API.
29 const [error, setError] = useState(''); //untuk menyimpan pesan error yang mungkin
30 // terjadi selama proses pengambilan data posts. Jika terjadi kesalahan saat mencoba
31 // mengambil data dari API, pesan error akan diperbarui dengan informasi yang relevan,
32 // dan pesan ini akan ditampilkan kepada pengguna untuk memberi tahu mereka tentang
33 // masalah yang terjadi saat mencoba memuat daftar postingan.
34
35 useEffect(() => { //untuk melakukan fetch data posts saat komponen pertama kali dimuat.
36 // Fungsi fetchPosts akan dipanggil dalam useEffect dengan array dependensi kosong,
37 // sehingga hanya akan dijalankan sekali saat komponen pertama kali dirender.
38 // Fungsi fetchPosts akan mencoba mengambil data posts dari API, mengelola status loading,
39 // dan menangani hasilnya untuk memperbarui state posts atau error sesuai dengan hasil pengambilan data.
40 fetchPosts(); //untuk melakukan fetch data posts saat komponen pertama kali dimuat.
41 // Fungsi fetchPosts akan dipanggil dalam useEffect dengan array dependensi kosong,
42 // sehingga hanya akan dijalankan sekali saat komponen pertama kali dirender.
43 // Fungsi fetchPosts akan mencoba mengambil data posts dari API, mengelola status loading,
44 // dan menangani hasilnya untuk memperbarui state posts atau error sesuai dengan hasil pengambilan data.
45 }, []);
46
47 const fetchPosts = async () => {
48   try {
49     setLoading(true);
50     const response = await postsAPI.getAll();
51     setPosts(response.data.data);
52     setError('');
53   } catch (err) {
54     setError('Gagal memuat posts. Silakan coba lagi.');
```

```

1     return (
2       <div className="min-h-screen bg-gray-100 flex flex-col">
3         <Navbar />
4
5         <main className="flex-1">
6           <div className="py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
7             <div className="mx-auto max-w-screen-sm text-center lg:mb-16 mb-8">
8               <h2 className="mb-4 text-3xl lg:text-4xl tracking-tight font-extrabold text-gray-900">
9                 Our Blog
10              </h2>
11              <p className="font-light text-gray-500 sm:text-xl">
12                Explore our collection of articles about React, web development, and more.
13              </p>
14            </div>
15
16            {loading ? (
17              <div className="flex justify-center items-center py-20">
18                <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
19              </div>
20            ) : error ? (
21              <div className="text-center py-20">
22                <p className="text-red-500 mb-4">{error}</p>
23                <button
24                  onClick={fetchPosts}
25                  className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5"
26                >
27                  Coba Lagi
28                </button>
29              </div>
30            ) : posts.length === 0 ? (
31              <div className="text-center py-20">
32                <p className="text-gray-500">Belum ada posts.</p>
33              </div>
34            ) : (
35              <div className="grid gap-8 lg:grid-cols-2">
36                {posts.map((post) => (
37                  <PostCard key={post.id} post={post} />
38                ))}
39              </div>
40            )}
41          </div>
42        </main>
43
44        <Footer />
45      </div>
46    );
47  };
48
49  export default PostList; //untuk mengekspor komponen PostList
50  // agar dapat digunakan di bagian lain dari aplikasi, terutama untuk digunakan
51  // sebagai rute daftar postingan dalam aplikasi React. Komponen ini bertanggung
52  // jawab untuk menampilkan daftar semua posts yang diambil dari API, serta
53  // mengelola status loading dan error selama proses pengambilan data.
54

```



E. Analisis dan Pembahasan

Pada Praktikum Modul 12 ini berhasil mengimplementasikan arsitektur decoupled modern dengan mengintegrasikan frontend berbasis React JS (Vite) dan backend REST API CodeIgniter 3. Melalui struktur folder yang modular, manajemen autentikasi global dikelola secara efisien menggunakan React Context API (AuthContext.jsx) untuk menghindari kendala props drilling dan memastikan data sesi pengguna sinkron di seluruh komponen. Keamanan navigasi pada sisi klien dijamin oleh library react-router-dom melalui komponen pembungkus ProtectedRoute.jsx. Komponen ini secara aktif mengevaluasi status login pengguna; jika akses ilegal atau tidak terautentikasi terdeteksi, sistem akan secara otomatis melakukan pengalihan (redirect) paksa ke halaman login. Di sisi komunikasi data, library Axios digunakan untuk menangani seluruh permintaan HTTP ke server backend. Implementasi Request Interceptor secara otomatis menyisipkan JWT (Bearer token) ke dalam header dokumen untuk setiap permintaan, sementara Response Interceptor menangani pembersihan data lokal secara otomatis jika server mengembalikan respons error 401 (Unauthorized). Selain itu, kendala Cross-Origin Resource Sharing (CORS) selama fase pengembangan berhasil diantisipasi dengan mengonfigurasi fitur reverse proxy pada vite.config.js. Jalur URL /ci3_project diarahkan langsung menuju target http://localhost, sehingga proses pertukaran data tetap aman dan lancar karena dianggap berasal dari asal (origin) yang sama.

F. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa pengembangan aplikasi Single Page Application (SPA) menggunakan React JS dan Vite telah berhasil diintegrasikan secara penuh dengan backend REST API CodeIgniter 3. Penerapan Global State Management melalui React Context API terbukti efektif dalam mengelola status autentikasi pengguna, yang kemudian dipadukan dengan komponen Protected Route dari react-router-dom untuk mengamankan hak akses halaman internal secara real-time.

Selain itu, konfigurasi interceptors pada Axios berhasil mengotomatisasi penyisipan token JWT ke dalam header request sekaligus menangani pembatalan sesi jika token kedaluwarsa sementara fitur reverse proxy pada Vite sukses mengatasi kendala pembatasan CORS selama proses pertukaran data lokal berlangsung.

G. Lampiran berisi Link repositori GitHub

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/ChalisaGavrillaAzhari-707012500127>